

Assembler language

Assembler instructions

Introduction

- So far in this unit, we have seen:
 1. The Assembler syntax
 2. How to reserve and initialize data using DC, DS, and literals
- In this section, we'll see a few more Assembler instructions
 - This will enable us to write our first working program

The beginning...

- All Assembler programs start with this statement:

Label	CSECT
-------	--------------

- CSECT stands for **Control Section**, an Assembler program unit
 - In other words, what we usually call “program”, the Assembler calls a Control Section
- The CSECT statement **must have a label**, which is the program name

...and the end

- The last statement in a program is:

END

- The END statement marks the end of the program
 - There are no statements after END (except a new CSECT, which is rare)
- The END statement **must not have a label**

The end - operand

- END has an ***optional*** operand:

END	[first executable instruction]
-----	--------------------------------

- When present, this operand points to the program's *first executable instruction*
- When omitted, the first executable instruction is the one at location 000000 (the first machine instruction following CSECT)
 - This is usually the case

CSECT and END - example

- Typically, a program looks like this:

DEM01	CSECT		
	LARL	12,DEM01	First instruction
	...	More instructions...	
INT1	DC	F'1'	Data definition
	...	More data definitions...	
	END		

- The program (CSECT) is called **DEM01**

CSECT and END - example

- Typically, a program looks like this:

DEM01	CSECT		
	LARL	12,DEM01	First instruction
	...	More instructions...	
INT1	DC	F'1'	Data definition
	...	More data definitions...	
	END		

- This is the program's first executable instruction...
 - (Ignore what the instruction does; we'll cover that soon)

CSECT and END - example

- Typically, a program looks like this:

DEM01	CSECT		
	LARL	12,DEM01	First instruction
	...	More instructions...	
INT1	DC	F'1'	Data definition
	...	More data definitions...	
	END		

- ...followed by the rest of the program's instructions

CSECT and END - example

- Typically, a program looks like this:

DEM01	CSECT		
	LARL	12,DEM01	First instruction
	...	More instructions...	
INT1	DC	F'1'	Data definition
	...	More data definitions...	
	END		

- Data definitions are usually placed after the instructions

CSECT and END - example

- Typically, a program looks like this:

DEM01	CSECT		
	LARL	12,DEM01	First instruction
	...	More instructions...	
INT1	DC	F'1'	Data definition
	...	More data definitions...	
	END		

- The last statement is END
 - Note no operand, as the first executable instruction is at the beginning of the program

The USING statement

- To understand USING, it helps to revisit **base-displacement**.
- Consider this instruction and the generated object code:

Location	Object Code	Source Statement
000006	48 F0 C018	LH 15,Int1 Load halfword into R15

Opcode

The USING statement

- To understand USING, it helps to revisit **base-displacement**.
- Consider this instruction and the generated object code:

Location	Object Code	Source Statement
000006	48F0 C018	LH 15,Int1 Load halfword into R15

Diagram illustrating the instruction and its generated object code:

- Object Code:** 48F0 C018
- Source Statement:** LH 15,Int1 Load halfword into R15

The object code is split into two parts: **Opcode** (48) and **1st Operand (R15)** (F0). The source statement is split into the instruction **LH** and the operand **15**.

The USING statement

- To understand USING, it helps to revisit **base-displacement**.
- Consider this instruction and the generated object code:

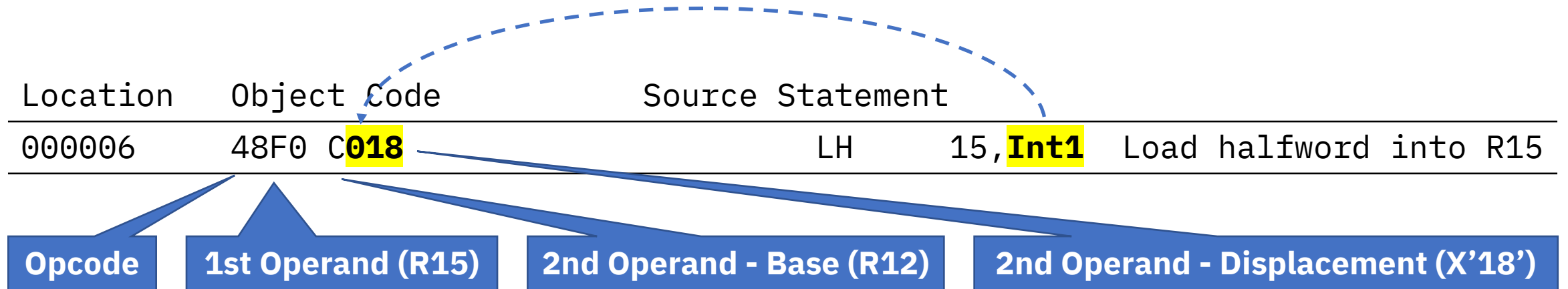
Location	Object Code	Source Statement
000006	48F0 C 018	LH 15, Int1 Load halfword into R15

Diagram illustrating the mapping of object code to source statement components:

- Opcode**: 48F0
- 1st Operand (R15)**: C
- 2nd Operand - Base (R12)**: 018

The USING statement

- To understand USING, it helps to revisit **base-displacement**.
- Consider this instruction and the generated object code:



The USING statement

- Two questions arise:
 1. The base register must be loaded with the correct address. How does that happen?
 2. There are 15 possible registers. How did the Assembler know which one to use?

The USING statement

- These instructions **load the base register** and **tell the Assembler to use it**:

DEM01	CSECT		
	LARL	12,DEM01	Load base register
	USING	DEM01,12	

- LARL (LOAD ADDRESS RELATIVE LONG) loads the program's starting address (DEM01) into register 12

The USING statement

- These instructions **load the base register** and **tell the Assembler to use it**:

DEM01	CSECT		
	LARL	12, DEM01	Load base register
	USING	DEM01, 12	

- LARL (LOAD ADDRESS RELATIVE LONG) loads the program's starting address (DEM01) into register 12
 - LARL doesn't need a base register to address DEM01, because it uses relative addressing.

The USING statement

- These instructions **load the base register** and **tell the Assembler to use it**:

DEMO1	CSECT		
	LARL	12, DEMO1	Load base register
	USING	DEMO1, 12	

- LARL (LOAD ADDRESS RELATIVE LONG) loads the program's starting address (DEMO1) in register 12
 - LARL doesn't need a base register to address DEMO1, because it uses relative addressing.
 - *LARL is the recommended way of loading a base register*

The USING statement

- These instructions **load the base register** and **tell the Assembler to use it**:

DEM01	CSECT		
	LARL	12, DEM01	Load base register
	USING	DEM01, 12	

- USING tells the Assembler two things:
 1. That the base address is DEM01
 2. To use register 12 as base register

Almost ready

- We are almost ready to write our first working program
- There are two more things we need to know
 - They are not related to Assembler or z/Architecture, but conventions of **z/OS**, the operating system where the program will run
- They are:
 - How to stop the program
 - How to pass a return code

Stopping the program

- In z/OS, register 14 contains the address to return to the program's caller
- Running “**BR 14**” (branch to the address in register 14) effectively stops the program
 - It is the last instruction to execute
 - Almost all Assembler programs running on z/OS end with “**BR 14**”

Passing a return code

- Like many operating systems, a z/OS program can pass a return code to indicate if the run was successful
- The return code is a value between 0 and 4095 **passed on register 15**
- For example, to set the return code to 20:

LH	15 ,=H'20'	Set RC=20
----	-------------------	-----------

Passing a return code

- The *convention* regarding return codes is:
 - Return code **0** means successful execution (no errors)
 - Return code **4** means warning (minor errors)
 - Return code **8** or higher means there were errors

A working program!

- What we know lets us write this program:

Location	Object Code	Source Statement
000000		DEM01 CSECT
000000	C0C0 0000 0000	LARL 12,DEM01 Load base reg.
		USING DEM01,12 Use R12 as base
000006	4830 C00E	LH 3,Int1 Load Int1 into R3
00000A	18F3	LR 15,3 Set return code
00000C	07FE	BR 14 End program
00000E	000A	Int1 DC H'10'
		END

- Let's looks at it in detail

Program start

Location	Object Code	Source	Statement
000000		DEM01	CSECT
		...	

- The program is called **DEM01**

Base register

Location	Object Code	Source Statement
000000		DEM01 CSECT
000000	C0C0 0000 0000	LARL 12,DEM01 Load base reg.
		USING DEM01,12 Use R12 as base
		...

- **LARL** loads the address of DEM01 into **register 12**
- **USING** tells the Assembler to use **register 12** as base register

Loading register

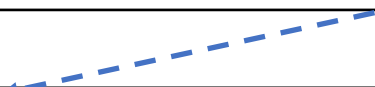
Location	Object Code	Source Statement		
...				
000006	4830 C00E	LH	3,Int1	Load Int1 into R3
...				
00000E	000A	Int1	DC	H'10'



- **LH** loads the halfword at **Int1** into **register 3**

Loading register

Location	Object Code	Source Statement		
...				
000006	4830 C00E	LH	3,Int1	Load Int1 into R3
...				
00000E	000A	Int1	DC	H'10'



- **LH** loads the halfword at **Int1** into **register 3**
- Note the base (register **12**) and displacement (**X'00E'**)
 - The displacement, in this case, coincides with the location, because the base register points to the start of the program (location **000000**)

Return code

Location	Object Code	Source Statement
...		
00000A	18F3	LR 15,3 Set return code
...		

- **LR** copies **register 3** (which contains decimal 10) to **register 15**
 - When the program ends, it will report a return code of **0010**

End of program execution

Location	Object Code	Source Statement
...		
00000C	07FE	BR 14 End program
...		
END		

- **BR 14** returns to caller (the operating system)
 - This ends the program execution

End of source program

Location	Object Code	Source Statement			
		...			
00000C	07FE		BR	14	End program
		...			
		END			

- **BR 14** returns to caller (the operating system)
 - This stops program execution
- **END** indicates the end of the *source program*
 - This tells the Assembler there are no more statements for this program

About this demo program

- This is, of course, a simple and not very useful demo program
- It shows a minimal Assembler program that produces just one output: a return code
- Although it works, it does not follow recommended practice
 - It misses parts that must be present in production programs
 - Those parts will be presented later

Program execution

- When the program runs and completes, the following message (shortened for readability) appears:

12.29.42 JOB34523 £HASP395 **DEM01 ENDED - RC=0010**

Program assembly

- We talked about “running the program”. How exactly does that happen?
- Our next topic explains how to assemble (compile) and run a program